

Сравнение на подходи за програмиране на захват и поставяне с mycobot_320 M5 в ROS2 / Gazebo Harmonic

Александър Тодоров · Докторантска разработка · Технически университет ·
Май 2026

ROS2 Humble · Gazebo Harmonic (gz-sim 8) · ros2_control · Ubuntu 22.04

Резюме

Настоящата разработка представя и сравнява два програмни подхода за реализация на задача „захват и поставяне“ (pick-and-place) с работна ръка mycobot_320 M5 (6 степени на свобода) в симулационна среда Gazebo Harmonic.

Подход А

използва ръчно определени, твърдо кодирани ставни ъгли, получени итеративно чрез приставката `rqt Joint Trajectory Controller`.

Подход В

използва аналитична Обратна Кинематика (IK), решена числено от библиотеката `ikru` въз основа на кинематичната верига от URDF файла на робота. Двата подхода са реализирани като самостоятелни ROS2 Humble пакети. Описани са методологията, изчисленията, кодът, резултатите и ограниченията на всеки подход, и е предоставено структурирано сравнение.

1. Въведение

Задачата „захват и поставяне“ е основна манипулационна операция в роботиката. Програмирането на работна ръка за изпълнение на тази задача изисква задаване на целеви конфигурации в *пространство на ставите* (ставни ъгли) или *Декартово пространство* (XYZ координати).

Двата подхода водят до принципно различни методологии с различни компромиси между простота и разширяемост. Тази разработка анализира тези компромиси чрез практическа реализация и документирана експериментална работа в симулация.

2. Описание на системата

2.1 Основни понятия за начинаещи

Ставни ъгли — всяка ставна позиция се описва с един ъгъл в радиани (rad):

$$0.0 \text{ rad} = 0^\circ \mid 1.57 \text{ rad} = 90^\circ \mid 3.14 \text{ rad} = 180^\circ \mid -1.57 \text{ rad} = -90^\circ$$

Формула: $\text{ъгъл}[^\circ] = \text{ъгъл}[\text{rad}] \times (180 / \pi) = \text{ъгъл}[\text{rad}] \times 57.296$

Координатна система — началото (0,0,0) е в центъра на основата на робота:

X → напред | Y → наляво | Z → нагоре (в метри)

ROS2 — рамка за разработка на роботни системи. Компонентите комуникират чрез *топици* (каналы за съобщения). Роботната ръка се управлява чрез публикуване на съобщение от тип `JointTrajectory` в топик `/arm_controller/joint_trajectory`.

Gazebo Harmonic — физичен симулатор, симулиращ гравитация, сблъсъци и сензори.

2.2 Роботна ръка `mycobot_320 M5`

Характеристика	Стойност
Степени на свобода	6 (ротационни стави)
Товароносимост	1 kg (реален хардуер)
Обхват	~320 mm
Захват	Адаптивен паралелен захват (4 пръста)
Симулация	Gazebo Harmonic, gz-sim 8
Управление	ros2_control: JointTrajectoryController + JointGroupPositionController

2.3 Дължини на звената (от URDF файла)

Основа → Ст.1: 173.9 mm Ст.1 → Ст.2: 88.78 mm Ст.2 → Ст.3: 135 mm		
Ст.3 → Ст.4: 120 mm Ст.4 → Ст.5: 95 mm Ст.5 → Ст.6: 65.5 mm	Ст.6 → Захват: ~144 mm	

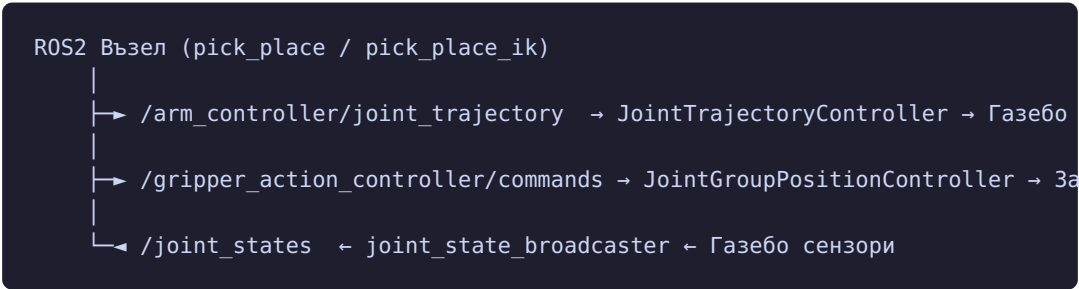
2.4 Именуване на ставите в ROS2

ROS2 Ставно Име	Физическа Става	Движение
joint2_to_joint1	J1 (основа)	Завъртане ляво/дясно (yaw)
joint3_to_joint2	J2 (рамо)	Наклон напред/назад (pitch)
joint4_to_joint3	J3 (лакът)	Наклон напред/назад (pitch)
joint5_to_joint4	J4 (китка 1)	Наклон (pitch)
joint6_to_joint5	J5 (китка 2)	Завъртане (roll)
joint6output_to_joint6	J6 (китка 3)	Завъртане (yaw)

2.5 Целеви обект — Куб

Характеристика	Стойност
Размери	40 mm (X) × 60 mm (Y) × 60 mm (Z)
Маса	10 g
Коефициент на триене	$\mu = 50$ (симулация)
Позиция в света	X=0.262 m, Y=-0.0842 m, Z=0.030 m
Ориентация	Завъртан на 90° (1.5708 rad)

2.6 Архитектура на управление



3. Подход А — Твърдо Кодирани Ставни Ъгли

3.1 Описание

При Подход А всяка целева позиция на робота се задава като фиксиран списък от шест ставни ъгли (в радиани). Тези ъгли са определени *ръчно* чрез приставката **rqt Joint Trajectory Controller**: плъзгачите се регулират докато ръката визуално се наравни с желаната позиция, след което стойностите се записват в кода.

3.2 Уейпойнти (Целеви Позиции)

Позиция	J1	J2	J3	J4	J5	J6	Цел
HOME	0.0004	0.0042	-0.0228	0.0045	0.0527	0.0077	Начална/ крайна поза
GRIPPER_READY	0.0	-0.0006	-0.0856	0.0226	1.5567	0.0	Захват вертикален, ръката напред
PRE_GRASP	0.0	-0.7050	-0.7237	-0.1689	1.5772	0.0	Над куба, готов за спускане
GRASP	0.0	-0.8150	-0.8340	-0.1689	1.5772	0.0	Захватът около куба

3.3 Изчисление на ъглите (радиани ↔ градуси)

$$\text{ъгъл}[^{\circ}] = \text{ъгъл}[\text{rad}] \times 57.296^{\circ}$$

PRE_GRASP J2: -0.7050 rad = **-40.4°** (рамо наклонено 40.4° напред)

PRE_GRASP J3: -0.7237 rad = **-41.5°** (лакът наклонен 41.5° напред)

GRASP J2: -0.8150 rad = **-46.7°** (с 6.3° по-дълбоко от PRE_GRASP)

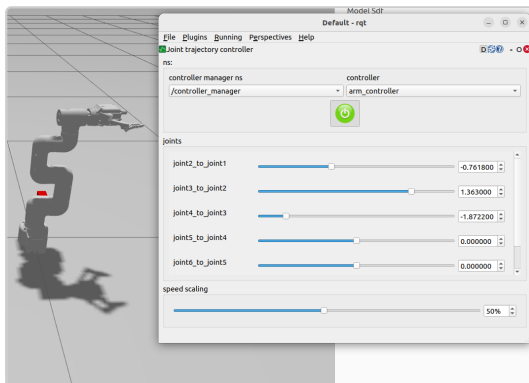
GRASP J3: -0.8340 rad = **-47.8°**

Разликата между PRE_GRASP и GRASP (~6°) съответства на ~20-25 mm спускане на крайния ефектор — точно необходимото за достигане на куба.

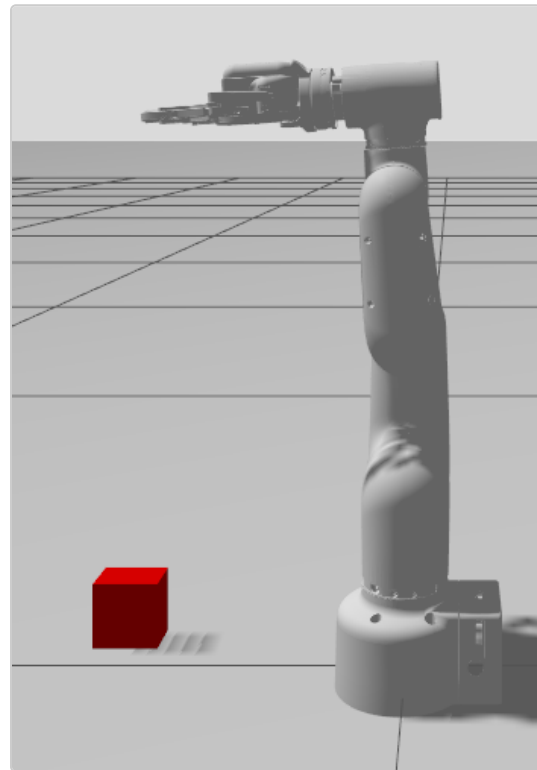
3.4 Методология — Как са намерени Ъглите

1. Стартиране на Gazebo симулацията с launch файла на Подход А
2. Отваряне на rqt → Plugins → Robot Tools → Joint Trajectory Controller

3. Активиране на `arm_controller` в приставката
4. Регулиране на плъзгачите за всяка става, докато ръката визуално се наравни
5. Записване на 6-те ставни стойности
6. Копиране в Python изходния код като константи
7. Стартиране на пълната последователност и корекция при нужда (~30-50 итерации)



Фиг. A1 — *rqt Joint Trajectory Controller*:
ръчно регулиране на ставни ъгли



Фиг. A2 — Позиция „захват готов“ (ръка
напред, захват вертикален)

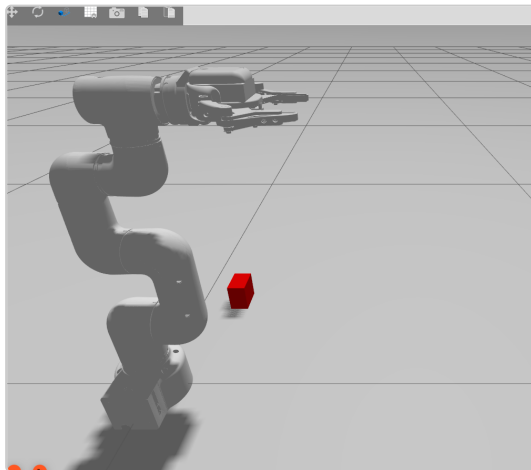
3.5 Код — Ключова Имплементация

```
HOME           = [+0.0004, +0.0042, -0.0228, +0.0045, +0.0527, +0.0077]
GRIPPER_READY  = [+0.0000, -0.0006, -0.0856, +0.0226, +1.5567, +0.0000]
PRE_GRASP      = [+0.0000, -0.7050, -0.7237, -0.1689, +1.5772, +0.0000]
GRASP          = [+0.0000, -0.8150, -0.8340, -0.1689, +1.5772, +0.0000]

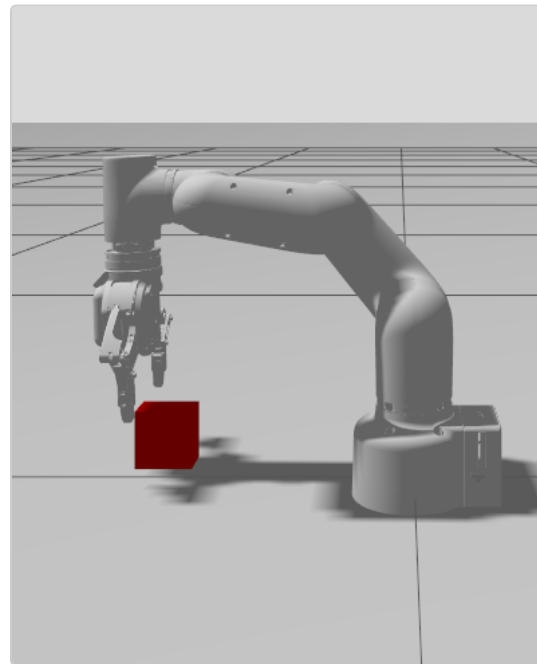
def _pick(self):
    self._grip(GRIPPER_OPEN)           # Отваряне на захвата
    self._move('home', HOME, sec=4)
    self._move('gripper_ready', GRIPPER_READY, sec=4)
    self._move('pre_grasp', PRE_GRASP, sec=4)
    self._move('grasp', GRASP, sec=5)
```

```
self._grip(GRIPPER_CLOSED)      # Затваряне на захвата
self._move('grasp', GRASP, sec=2) # Задържане след захват
self._wait(1.0)
self._grip(GRIPPER_CLOSED)      # Повторна команда за захват
self._wait(0.3)
self._move('pre_grasp', PRE_GRASP, sec=5) # Повдигане
self._move('home', HOME, sec=6)
```

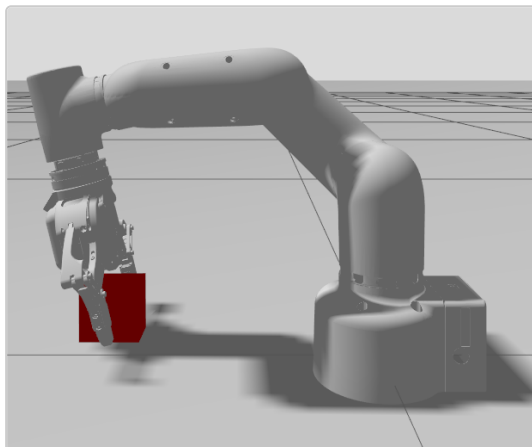
3.6 Скриншотове — Подход А



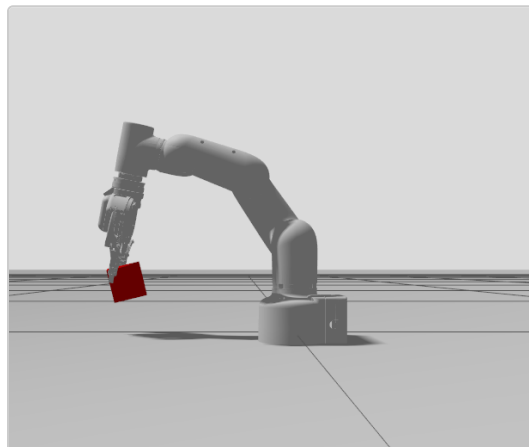
Фиг. А3 — HOME позиция (начална)



Фиг. А4 — PRE_GRASP (над куба)



Фиг. А5 — GRASP (захватът около куба)



Фиг. А6 — LIFT (повдигане)

```

alexander@alexander-Z370M-D53H:~/Projects/mycobot_ws$ source /opt/ros/humble/setup.bash && source ~/Projects/mycobot_ws/install/setup.bash && ros2 run approach_a pick_place
[INFO] [1778792996.418049999] [pick_place]: Waiting for joint states...
[INFO] [1778792997.420873580] [pick_place]: Gripper: OPEN (0.0)
[INFO] [1778792997.522576713] [pick_place]: Moving to: home
[INFO] [1778792997.576825235] [pick_place]: Reached: home
[INFO] [1778792999.617719256] [pick_place]: Moving to: gripper_ready
[WARN] [1778793019.632787621] [pick_place]: Timeout reaching: gripper_ready - continuing anyway
[INFO] [1778793019.636315887] [pick_place]: Moving to: pre_grasp
[WARN] [1778793019.651767081] [pick_place]: Timeout reaching: pre_grasp - continuing anyway
[INFO] [1778793039.657603272] [pick_place]: Moving to: grasp
[WARN] [1778793059.672612344] [pick_place]: Timeout reaching: grasp - continuing anyway
[INFO] [1778793059.674235251] [pick_place]: Grippers: CLOSE (-1.0)
[INFO] [1778793059.777610409] [pick_place]: Moving to: grasp
[WARN] [1778793079.829111197] [pick_place]: Timeout reaching: grasp - continuing anyway
[INFO] [1778793080.834067848] [pick_place]: Gripper: CLOSE (-1.0)
[INFO] [1778793081.236080565] [pick_place]: Moving to: pre_grasp
[WARN] [1778793101.249996500] [pick_place]: Timeout reaching: pre_grasp - continuing anyway
[INFO] [1778793101.252096832] [pick_place]: Moving to: home
[INFO] [1778793101.305512096] [pick_place]: Reached: home
[INFO] [1778793103.308261275] [pick_place]: Done.
  
```

Фиг. А7 — Терминален изход на последователността на Подход А

► **Видео:** Пълна последователност на Подход А — виж [approaches/approach_A/screenshots/A_full_sequence.webm](#)

3.7 Предимства

- ✓ Изключително проста имплементация — не се изискват математически библиотеки
- ✓ Напълно детерминистичен — едни и същи ъгли при всяко изпълнение
- ✓ Лесен за отстраняване на грешки — вижда се точно коя става е грешна
- ✓ Не изисква URDF анализ по време на изпълнение
- ✓ Работи с всеки модел роботна ръка

3.8 Ограничения

- ✗ Всяка нова целева позиция изисква ръчна рекалибрация (~30-50 rqt итерации)
- ✗ Не е преизползваем — ъглите са специфични за точното местоположение на куба

- ✗ Липса на Декартова интуиция — трудно е да се разсъждава в пространство XYZ
- ✗ Не може да работи в динамични среди, където обектите се движат
- ✗ Необходима е допълнителна междинна поза GRIPPER_READY за правилна конфигурация

4. Подход В — Обратна Кинематика (ikpy)

4.1 Описание

При Подход В целевите позиции се задават в **Декартово пространство** (X, Y, Z в метри). Кинематичната верига на робота се анализира при стартиране от библиотеката **ikpy**, която числено решава ставните ъгли, поставящи крайния ефектор в желаната позиция. Това елиминира необходимостта от ръчна калибрация.

4.2 Права Кинематика (FK — Forward Kinematics)

FK изчислява позицията на крайния ефектор от ставните ъгли:

$$T_{06} = T_{01} \cdot T_{12} \cdot T_{23} \cdot T_{34} \cdot T_{45} \cdot T_{56}$$

Всяка T_{ij} е 4x4 хомогенна трансформационна матрица:

$$T_{ij} = \text{Rot}(\text{oc}, \theta) \cdot \text{Trans}(dx, dy, dz)$$

Всяка матрица кодира ротацията (ставен ъгъл θ) и трансляцията (дължина на звеното) между последователни координатни системи.

4.3 Обратна Кинематика (IK — Inverse Kinematics)

IK обръща задачата: при дадена желана позиция $p = [x, y, z]$, намери ставните ъгли θ :

$$\theta^* = \text{argmin} \parallel \text{FK}(\theta) - p_{\text{цел}} \parallel^2$$

Решава се числено от **ikpy** чрез оптимизационен алгоритъм (L-BFGS-B / градиентно спускане).
Конвергенцията зависи от началното приближение (seed).

4.4 Калибрация на Отместването на Инструмента

URDF кинематичната верига завършва при *link6*. Действителният център на захвата е ~144 mm под *link6* по локалната Z-ос, плюс малки XY отмествания, положени от механична асиметрия. Това отместване е калибрирано с ъглите на GRASP от Подход А като опорна точка:

$$\begin{aligned} FK(\theta_{\text{grasp_A}}) &\rightarrow T_{\text{link6_световни_координати}} \\ T00L_OFFSET_LOCAL &= T_{\text{link6}}^{-1} \cdot p_{\text{куб}} \\ \text{Резултат: } T00L_OFFSET_LOCAL &= [-0.0032, -0.0033, 0.1440] \text{ m} \end{aligned}$$

Отместването се подава на iKру като `last_link_vector`, удължавайки веригата.

4.5 Уейпойнти (Декартови Координати)

Позиция	X (m)	Y (m)	Z (m)	Цел
PRE_GRASP	0.2620	-0.0842	0.1200	90 mm над горната страна на куба
GRASP	0.2620	-0.0842	0.0450	15 mm над основата на куба
LIFT	0.2620	-0.0842	0.1500	120 mm над основата на куба

Позиция на куба: X=0.262 m, Y=-0.0842 m, Z=0.030 m (основа в симулацията)

4.6 Стратегия за Начално Приближение (Seed)

Тъй като IK има множество решения, *начално приближение* насочва решавача към правилната конфигурация на робота:

Позиция	Използвано Seed	Причина
PRE_GRASP	Ъгли PRE_GRASP от Подход А	Желана е същата физическа конфигурация
GRASP	Ъгли GRASP от Подход А	Осигурява правилна посока на подход
LIFT	Решените ъгли на GRASP	Приемственост — ръката остава в същата конфигурация

Отстранена критична грешка: Първоначално LIFT използваше твърдо кодираните ъгли на GRASP от Подход А като seed. След като IK решава GRASP числено, действителните стойности се различават. Грешният seed

накара IK да намери огледална конфигурация ($J_5 \approx 2.24$ вместо 1.58 rad), което причини промахване и timeout. Поправка: използване на действително решените ъгли на GRASP като seed за LIFT.

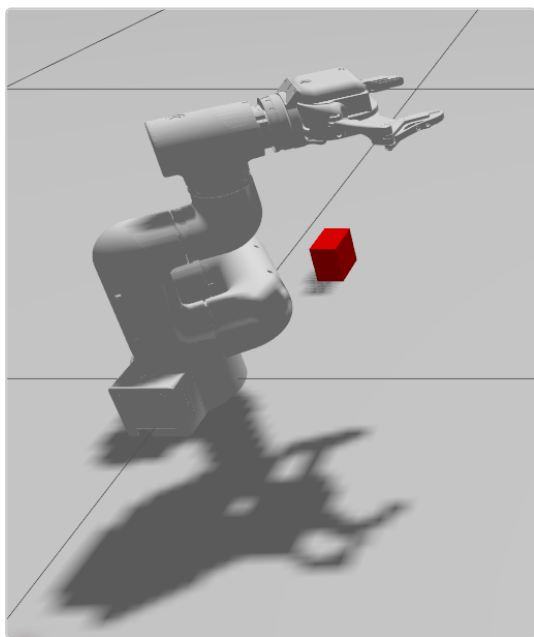
4.7 Код — Изграждане на Кинематична Верига

```
def build_chain(urdf_path):
    chain_full = ikpy.chain.Chain.from_urdf_file(
        urdf_path,
        base_elements=['world'],
        last_link_vector=TOOL_OFFSET_LOCAL,    # [-0.0032, -0.0033, 0.144]
    )
    # Отрязване при joint6output за избягване на разклонение на захвата
    stop_idx = next(i for i, l in enumerate(chain_full.links)
                    if l.name == 'joint6output_to_joint6')
    arm_links = list(chain_full.links[:stop_idx + 1]) + [chain_full.links[-1]]
    chain = ikpy.chain.Chain(arm_links)
    chain.active_links_mask = [l.name in set(ARM_JOINTS) for l in chain.links]
    return chain
```

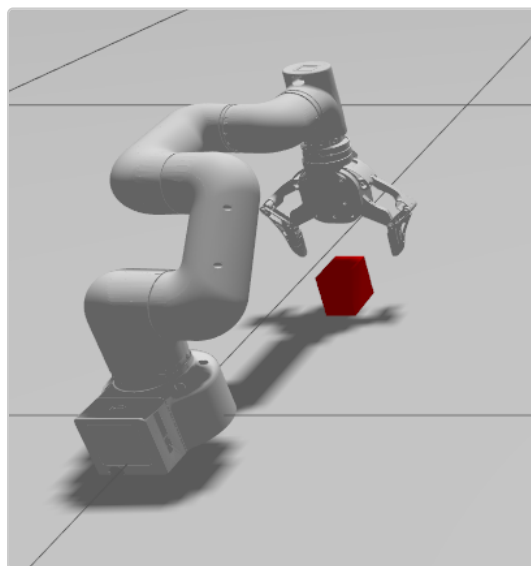
4.8 Код — Решаване на IK

```
def ik_solve(chain, arm_indices, target_xyz, seed_joints):
    seed = np.zeros(len(chain.links))
    for idx, val in zip(arm_indices, seed_joints):
        seed[idx] = val
    result = chain.inverse_kinematics(
        target_position=target_xyz,
        initial_position=seed,
        orientation_mode=None,    # само позиция, без ориентация
    )
    joints = [result[i] for i in arm_indices]
    joints[-1] = 0.0    # принудително J6=0 — захватът остава перпендикулярен
    return joints
```

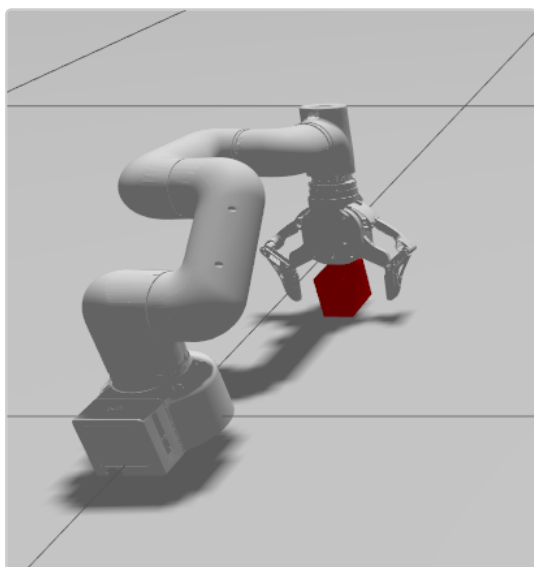
4.9 Скриншотове — Подход В



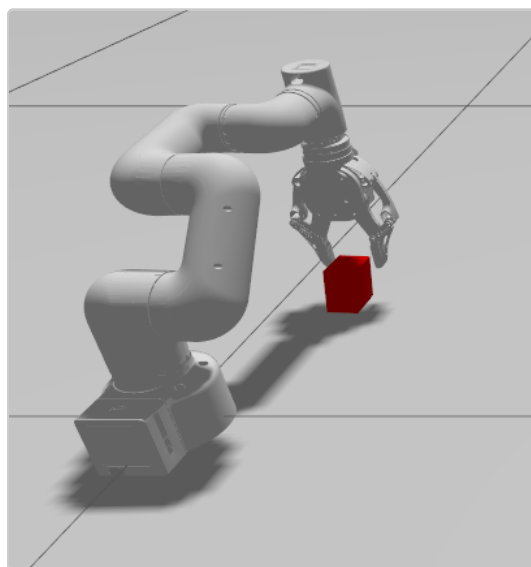
Фиг. В1 — HOME позиция (автоматично при стартиране)



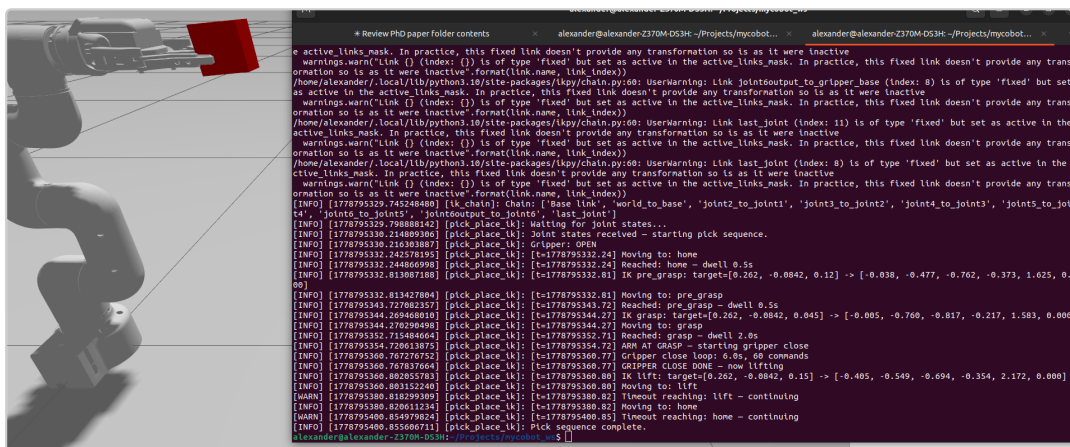
Фиг. В2 — PRE_GRASP (IK решение: $X=0.262$, $Y=-0.084$, $Z=0.120$)



Фиг. В3 — GRASP (IK решение: $Z=0.045$)



Фиг. В4 — LIFT (IK решение: $Z=0.150$)



Фиг. В5 — Терминален изход с IK решените ставни ъгли и пълния лог

► **Видео:** Пълна последователност на Подход В — виж [approaches/approach_B/screenshots/B_full_sequence.webm](#)

4.10 Предимства

- ✓ Уейпointите се задават в интуитивни Декартови (XYZ) координати
- ✓ Нова целева позиция изисква само смяна на три числа (X,Y,Z)
- ✓ Разширяем — добавянето на нови позиции е тривиално
- ✓ Основа за управление от сензори (камера → XYZ → IK)
- ✓ Автоматична начална поза при стартиране (set_home_pose)

4.11 Ограничения

- ✗ Изисква URDF и зависимост iкру
- ✗ IK има множество решения — грешно начално приближение → грешна конфигурация
- ✗ Отместването на инструмента трябва да бъде калибрирано от известна опорна точка
- ✗ IK само за позиция: без ограничение на ориентацията (ъгълът на захвата не се контролира)
- ✗ Същото ограничение на Gazebo физиката: захватът се хлъзга при динамично повдигане

5. Сравнение на Подходите

Критерий	Подход А (Твърдо кодиран)	Подход В (Обратна Кинематика)
----------	---------------------------	-------------------------------

Задаване на позиции	Ставни ъгли [rad]	Декартови координати XYZ [m]
Усилие за настройка	30-50 rqt итерации на поза	Само зададени XYZ, IK автоматично
Математика	Не се изисква (визуално)	FK/IK, хомогенни трансформации
Зависимости	rcipy, trajectory_msgs	+ ikpy, numpy, URDF
Детерминизъм	100% еднакво при всяко изпълнение	Детерминистично при фиксирано seed
Нова целева поза	Ръчна рекалибрация	Смяна на 3 числа (X,Y,Z)
Ориентация на захвата	Контролирана чрез J5	Не е ограничена (orientation_mode=None)
Сложност на кода	Прост (~100 реда)	Умерена (~250 реда)
Нужни междинни пози	Да (GRIPPER_READY)	Не — IK управлява конфигурацията
Мащабируемост	Лоша — ръчна за всяка поза	Добра — един IK разговор на поза
Готовност за сензорна интеграция	Не	Да (XYZ от камера)
Физика на захвата (симулация)	Ограничена (Gazebo)	Ограничена (същата Gazebo)

6. Известни Проблеми и Решения

Проблем	Подход	Приложено Решение
Ръката не е в начална поза при стартиране	В	Добавен set_home_pose след зареждане на arm_controller
Грешно IK решение за LIFT ($J5 \approx 2.24$)	В	Използване на решените ъгли на GRASP като seed вместо твърдо кодирани
Контролерите се провалят при повторно стартиране	И двата	Убиване на всички ROS2/Gazebo процеси преди рестарт
Конфликт на дублиращи имена на пакети	И двата	COLCON_IGNORE в директорията paper/

Захватът се хлъзга при повдигане	И двата	Документирано като ограничение на Gazebo физиката
Ръката не достига дълбочината за захват	A	Итеративна rqt калибрация: J2 регулиран от -0.705 до -0.815 rad

7. Как да Стартирате Кода

Подход А

```
# Компилиране
colcon build --packages-select approach_a --symlink-install
source install/setup.bash

# Стартиране на симулацията (ръката автоматично отива в HOME)
ros2 launch approach_a gz_harmonic.launch.py

# Изпълнение на захват (в нов терминал)
ros2 run approach_a pick_place
```

Подход В

```
# Инсталиране на IK зависимост
pip install ikpy

# Компилиране
colcon build --packages-select pick_place_ik --symlink-install
source install/setup.bash

# Стартиране на симулацията (ръката автоматично отива в HOME)
ros2 launch pick_place_ik gz_harmonic_ik.launch.py

# Изпълнение на захват (в нов терминал)
ros2 run pick_place_ik pick_place_ik
```

8. Заключение

И двата подхода успешно демонстрират задачата „захват и поставяне“ в симулационната среда Gazebo Harmonic.

Подход А е препоръчителна отправна точка за начинаещи: не изисква математика, лесен е за отстраняване на грешки и дава надеждни резултати след калибрация. Основното ограничение е мащабируемостта — всяка нова позиция изисква нов цикъл на ръчна калибрация.

Подход В решава проблема с мащабируемостта като работи в Декартово пространство. Нова целева позиция изисква само смяна на три координатни стойности. Това е естествената основа за управление от сензори — камерата предоставя XYZ позиция на обекта в реално време, а IK автоматично изчислява необходимите ставни ъгли. Допълнителната сложност (URDF анализ, IK библиотека, управление на начално приближение, калибрация на отместване) е оправдана за приложения с повече от една позиция за захват или за динамични среди.

И двата подхода споделят едно и също фундаментално ограничение на симулацията: контактният модел на Gazebo не осигурява достатъчно триене за надеждно задържане на куба при динамично ускорение на ръката. Производствено решение би изисквало или приставка за прикрепяне на захват (grasp-attachment plugin), или PID-управляван захват с обратна връзка по сила.

9. Литература

1. Elephant Robotics. *myscobot_320 M5 Техническа Спецификация*. 2022.
2. Open Robotics. *ROS2 Humble Документация*. docs.ros.org, 2022.
3. Open Robotics. *ros2_control: JointTrajectoryController*. GitHub, 2022.
4. Mayer, B. *ikpy: Обратна Кинематика за Python*. GitHub: Phylliade/ikpy, 2016.
5. Open Robotics. *Gazebo Harmonic (gz-sim 8) Документация*. gazebosim.org, 2023.
6. Denavit, J., Hartenberg, R.S. *Кинематична нотация за механизми с нисши двойки*. ASME Journal, 1955.